

AWS CLOUD ARCHITECTURE & DEVOPS

Deploy Spring Boot on AWS Elastic Beanstalk

Production-ready deployment guide with Application Load Balancer, EC2 Auto Scaling, IAM Security Roles, and decoupled Amazon RDS.

1. Cloud Architecture Overview

This guide demonstrates how to deploy a scalable, fault-tolerant Spring Boot web service to **AWS Elastic Beanstalk**. By combining Elastic Beanstalk's managed platform with Layer-7 Application Load Balancing (ALB), multi-AZ EC2 Auto Scaling, and Amazon Relational Database Service (RDS), you ensure continuous availability and automated scaling under high traffic loads.

1. AWS Elastic Beanstalk

- Fully managed PaaS abstracting server provisioning and operating system patching.
- Supports Java 21 on Amazon Linux 2023 with built-in health monitoring.
- Automates zero-downtime rolling code updates.

2. Amazon EC2 (Elastic Compute Cloud)

- Resizable compute capacity hosting the Spring Boot runtime JAR.
- Flexible instance types (e.g., **t3.micro** for Free Tier efficiency).
- Integrated with IAM Instance Profiles for least-privilege cloud security.

3. Auto Scaling Group (ASG)

- Dynamically adds or removes EC2 instances based on traffic demands.
- Configured with CPU utilization breach thresholds (80% scale-out, 40% scale-in).
- Enforces minimum (1) and maximum (4) instance fleet boundaries.

4. Elastic Load Balancer (ALB)

- Dedicated Layer-7 Application Load Balancer distributing HTTP port 80 traffic.
- Continuous target health-check probes on the Spring Boot **/** endpoint.
- Spans multiple Availability Zones (AZs) for high availability.

Architectural Best Practice (Decoupled Amazon RDS): Never provision an integrated database directly inside the Elastic Beanstalk environment. An in-environment database shares the lifecycle of the Beanstalk stack; terminating or rebuilding the environment will irrevocably destroy the database. Instead, deploy a standalone Amazon RDS instance and connect your Spring Boot application securely using Environment Properties.

2. Step-by-Step Deployment Walkthrough

01 Navigating to Elastic Beanstalk Console

Sign in to the AWS Management Console, open **Elastic Beanstalk** from the Services menu, and click **Create application**.

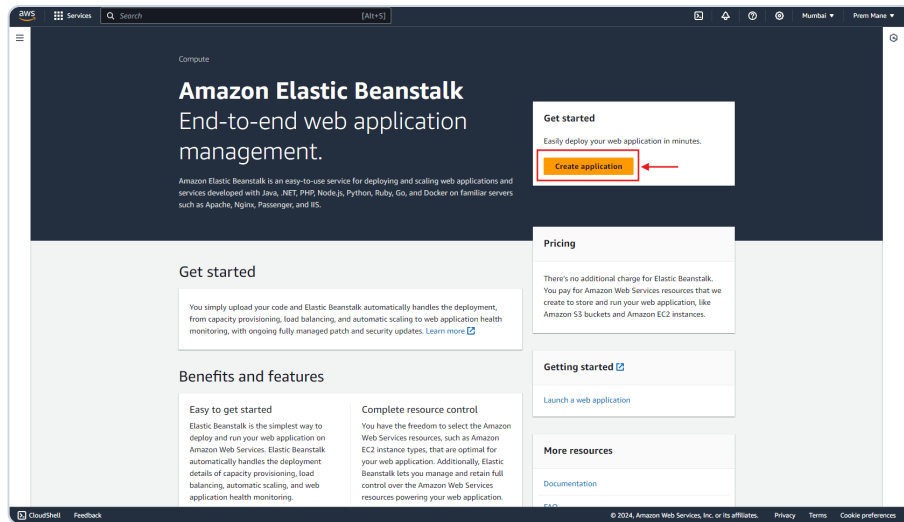


Figure 1: Elastic Beanstalk console landing page.

02

Step 1: Configure Environment, Platform & Presets

Define your core application namespace, platform runtime, and initial instance preset:

- **Environment Tier:** Select **Web server environment**.
- **Application Name:** Enter **Library-management-service**.
- **Environment Name:** Enter **Library-management-service-env** and verify subdomain availability.
- **Platform:** Choose **Managed platform > Java > Corretto 21 running on 64bit Amazon Linux 2023**.
- **Application Code:** Select **Sample application** for initial setup.
- **Configuration Presets:** Select **Single instance (free tier eligible)** (we enable Load Balancing in Step 4).

Figure 2: Environment tier & subdomain setup.

Figure 3: Platform and sample code selection.

Figure 4: Selecting Single instance preset before proceeding.

03

Step 2: Configure Service Access & IAM Security Roles

Elastic Beanstalk and EC2 instances require dedicated IAM roles to interact with AWS services:

- **Service Role:** Choose **Create and use new service role** (named **aws-elasticbeanstalk-service-role**).
- **EC2 Instance Profile:** Select **aws-elasticbeanstalk-ec2-role** from the dropdown.

Figure 5: Setting Beanstalk service role and EC2 instance profile.

Creating the EC2 IAM Role: If **aws-elasticbeanstalk-ec2-role** is not present, navigate to the **IAM Console > Roles > Create role**. Choose **AWS service > EC2**, attach the managed policy **AWSElasticBeanstalkCustomPlatformforEC2Role** (or **AWSElasticBeanstalkWebTier**), name it **aws-elasticbeanstalk-ec2-role**, and click **Create role**.

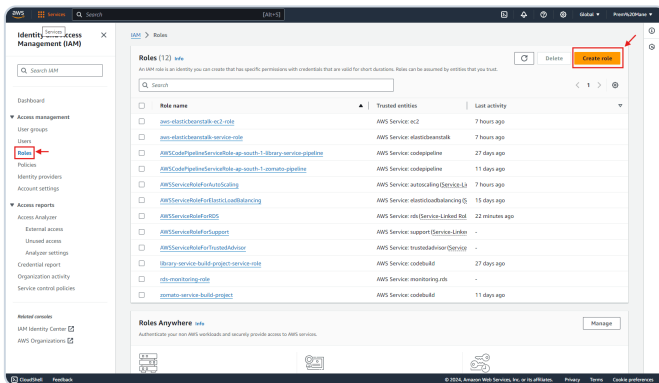


Figure 6: IAM Console — Create Role.

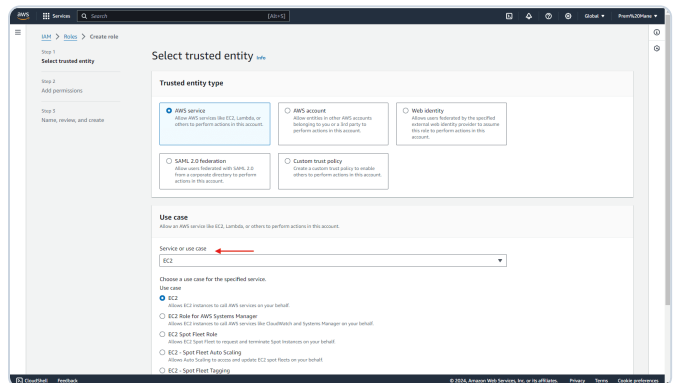


Figure 7: Trusted entity & EC2 use case.

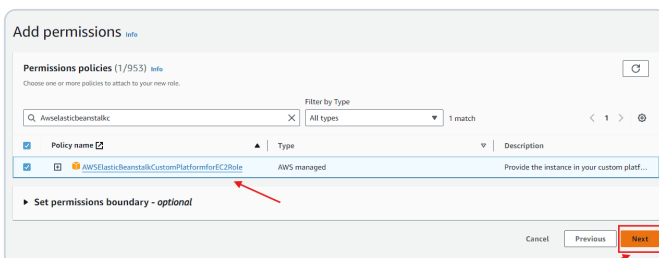


Figure 8: Attaching managed EC2 role policy.

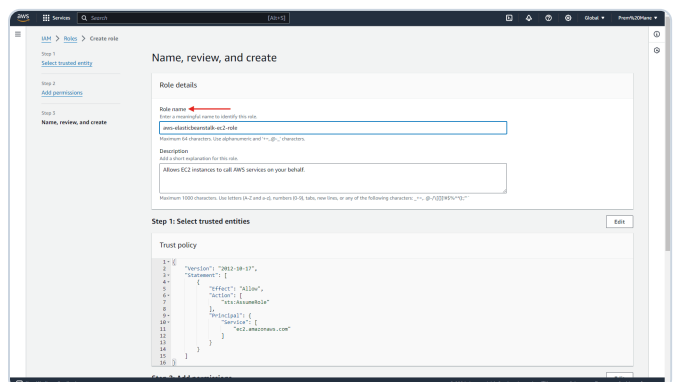


Figure 9: Setting role name & trust policy.

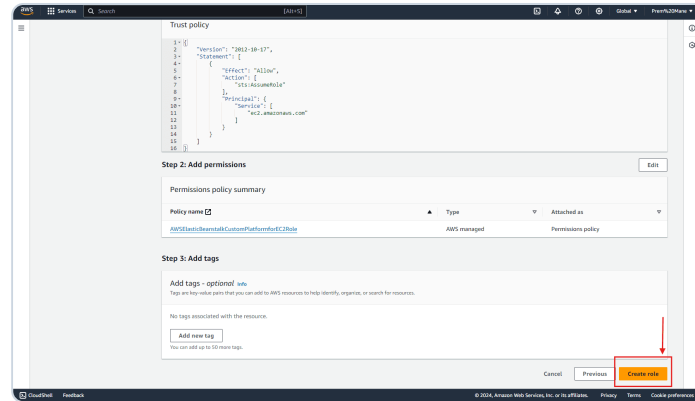


Figure 10: Finalizing IAM role creation in AWS Identity & Access Management.

04

Step 3: Networking, Database Decoupling & Tags

Configure VPC isolation and distribute instances across multiple Availability Zones for high availability:

- **Virtual Private Cloud (VPC):** Select your default or custom VPC.
- **Public IP Address:** Check **Activated** under Instance settings to grant internet-routable public IPs.
- **Instance Subnets:** Check all availability zone subnets (e.g., **ap-south-1a**, **ap-south-1b**, **ap-south-1c**).
- **Database Section:** Leave *completely unconfigured* to maintain independent Amazon RDS persistence.

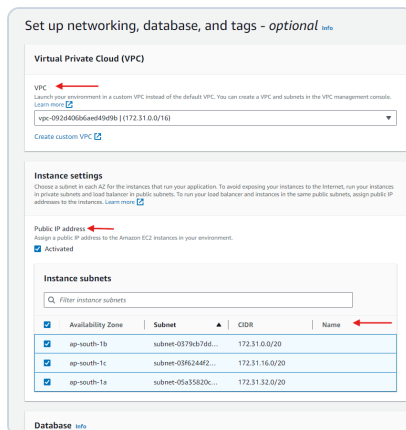


Figure 11: VPC network and multi-AZ subnets.

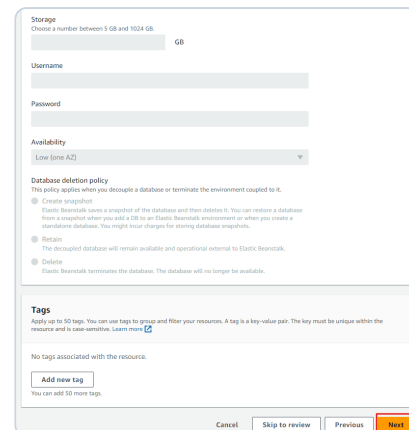


Figure 12: Bypassing the integrated database.

05

Step 4: Auto Scaling, Load Balancer & Health Checks

Configure your production compute fleet, auto scaling metric alarms, and the Application Load Balancer:

Capacity

Configure the compute capacity of your environment and auto scaling settings to optimize the number of instances used.

Auto scaling group

Environment type
Select a single instance or load-balanced environment. You can develop and test an application in a single-instance environment to save costs and then upgrade to a load-balanced environment when the application is ready for production. [Learn more](#)

Load balanced

Instances
Min: 1
Max: 4

Fleet composition
Spot instances are launched at the lowest available price. [Learn more](#)

☒ On-Demand instances
☐ Combine purchase options and instances

Maximum spot price
The maximum price per instance-hour, in USD, that you're willing to pay for a Spot Instance. Setting a custom price limits your chances to fulfill your target capacity using Spot instances.
☐ Default
☒ Set your maximum price

On-Demand base
The minimum number of On-Demand instances that your Auto Scaling group provisions before considering Spot instances as your environment scales out.
0

On-Demand above base
The percentage of On-Demand instances as part of any additional capacity that your Auto Scaling group provisions beyond the On-Demand base instances.
0 %

Capacity rebalancing
Specify whether to enable the capacity rebalancing feature for Spot instances in your Auto Scaling Group. This option is only relevant

Figure 13: Setting Load balanced type (Min: 1, Max: 4).

Architecture
The processor architecture determines the instance types that are made available. You can't change this selection after you create the environment. [Learn more](#)

☒ x86_64
This architecture uses x86 processors and is compatible with most third-party tools and libraries.

☐ arm64 - new
This architecture uses ARM's Graviton2 processors. You might have to recompile some third-party tools and libraries.

Instance types
Add instance types for your fleet. Change the order that the instances are in to set the preferred launch order. This only affects On-Demand instances. We recommend you include at least two instance types. [Learn more](#)

Choose x86 instance types

t3.micro

AMI ID
Elastic Beanstalk selects a default Amazon Machine Image (AMI) for your environment based on the Region, platform version, and processor architecture that you choose. [Learn more](#)

ami-02fca4f5346293cc2

Availability Zones
Number of Availability Zones (AZs) to use.
Any

Placement
Specify Availability Zones (AZs) to use.
Choose Availability Zones (AZs)

Scaling cooldown
300 seconds

Scaling triggers

Metric
Change the metric that is monitored to determine if the environment's capacity is too low or too high.
CPUUtilization

Figure 14: Choosing x86_64 and t3.micro compute.

Auto Scaling Metric Parameters

Scaling Trigger Parameter	Configured Value	Operational Behavior
Metric & Statistic	CPUUtilization / Average	Monitors average host CPU utilization across the fleet
Evaluation Period & Duration	1 Min / 5 Min breach duration	Prevents premature scaling from transient spikes
Upper Threshold	80% (+1 EC2 instance)	Automatically scales out capacity under elevated load
Lower Threshold	40% (-1 EC2 instance)	Automatically scales in capacity to reduce running costs

Scaling triggers

Metric
Change the metric that is monitored to determine if the environment's capacity is too low or too high.
CPUUtilization

Statistic
Choose how the metric is interpreted.
Average

Unit
Percent

Period
The period between metric evaluations.
1 Min

Breach duration
The amount of time a metric can exceed a threshold before triggering a scaling operation.
5 Min

Upper threshold
80

Scale up increment
1 EC2 instances

Lower threshold
40 capacity

Scale down increment
-1 EC2 instances

Figure 15: Configuring CPUUtilization metrics and breach threshold rules.

Dedicated Application Load Balancer (ALB)

- **Visibility:** Select **Public**.
- **Subnets:** Check all availability zones for balanced traffic routing.
- **Load Balancer Type:** Choose **Application load balancer** and **Dedicated**.
- **Listeners:** Configure port **80** with **HTTP** listener enabled.

Load balancer network settings

Visibility
Make your load balancer internal if your application serves requests only from connected VPCs. Public load balancers serve requests from the Internet.

Public

Load balancer subnets

Filter load balancer subnets

	Availability Zone	Subnet	CIDR	Name
☒	ap-south-1b	subnet-0379cb7...	172.31.0.0/20	
☒	ap-south-1c	subnet-03f6244f...	172.31.16.0/20	
☒	ap-south-1a	subnet-05a3582...	172.31.32.0/20	

Figure 16: Public ALB visibility & subnets.

Load Balancer Type

☒ **Application load balancer**
Application layer load balancer—routing HTTP and HTTPS traffic based on protocol, port, and route to environment processes.

☐ **Classic load balancer**
Previous generation — HTTP, HTTPS, and TCP

☐ **Network load balancer**
Ultra-high performance and static IP addresses for your application.

☒ **Dedicated**
Use a load balancer that Elastic Beanstalk creates exclusively for this environment.

☐ **Shared**
Use a load balancer that someone in your account created. It can be shared among multiple Elastic Beanstalk environments.

Listeners
You can specify listeners for your load balancer. Each listener routes incoming client traffic on a specified port using a specified protocol to your environment processes. By default, we've configured your load balancer with a standard web server on port 80.

Listener Port	Listener Protocol	SSL certificate	Default process	Enabled
80	HTTP	—	default	☒

Figure 17: Dedicated ALB and HTTP listener.

Spring Boot Health Check Endpoint

The ALB periodically probes the health check path (default `/`) on port 80. Your Spring Boot application must expose a REST controller that responds with HTTP 200 OK:

```
package com.library.controller; import org.springframework.http.ResponseEntity; import
org.springframework.web.bind.annotation.GetMapping; import
org.springframework.web.bind.annotation.RestController; @RestController public class
HealthCheckController { @GetMapping("/") public ResponseEntity<String> healthCheckController() {
return ResponseEntity.ok("Ok"); } }
```

Processes
For each environment process, you can specify the protocol and port that the load balancer uses to route requests to the process. You can also specify how the load balancer performs process health checks.

Name	Port	Protocol	HTTP code	Health check path	Stickiness
default	80	HTTP		/	Disabled

Rules
Your load balancer routes requests to environment processes based on rules. Rules are evaluated by priority in ascending numerical order. Elastic Beanstalk configures a default rule for each listener. Each default rule routes all traffic to the default process associated with each listener, and has the last priority among all rules of that listener. If a request doesn't match the conditions for any other rule, a default rule routes the request to the listener's default process.

No additional listener rules are currently configured.
Click Add rule to add a listener rule.

Figure 18: Default process on port 80 with "/" path.

Rules
Your load balancer routes requests to environment processes based on rules. Rules are evaluated by priority in ascending numerical order. Elastic Beanstalk configures a default rule for each listener. Each default rule routes all traffic to the default process associated with each listener, and has the last priority among all rules of that listener. If a request doesn't match the conditions for any other rule, a default rule routes the request to the listener's default process.

No additional listener rules are currently configured.
Click Add rule to add a listener rule.

Log files access
Configure Elastic Load Balancing to capture logs with detailed information about requests sent to your Load Balancer. Logs are stored in Amazon S3.

Store logs
Standard Amazon S3 charges apply.
☒ Enabled

S3 Bucket
S3 Bucket

Prefix
Standard Amazon S3 charges apply.

Cancel Skip to review Previous Next

Figure 19: Listener rules and S3 log access options.

06

Step 5: Monitoring, Deployments & Environment Properties

Configure operational oversight and supply required Spring Boot environment variables:

- **Health Reporting:** Select **Basic** monitoring.
- **Deployment Policy:** Choose **Rolling** to ensure continuous uptime during application updates.
- **Environment Properties:** Supply your database credentials and runtime properties as detailed below:

Monitoring info

Health reporting
Enhanced health reporting provides free real-time application and operating system monitoring of the instances and other resources in your environment. The `EnvironmentHealth` custom metric is provided free with enhanced health reporting. Additional charges apply for each custom metric. For more information, see [Amazon CloudWatch Pricing](#).

System
☒ Basic
☐ Enhanced

Health event streaming to CloudWatch Logs
Configure Elastic Beanstalk to stream environment health events to CloudWatch Logs. You can set the retention up to a maximum of ten years and configure Elastic Beanstalk to delete the logs when you terminate your environment.

Log streaming
☒ Activated (standard CloudWatch charges apply.)

Retention
7

Lifecycle
Keep logs after terminating environment

Figure 20: Basic health monitoring and CloudWatch log streaming settings.

Property Name	Configured Value / Operational Purpose
DB_HOST_URL	postgres-spring-1.c7sgj40a0vdq.ap-south-1.rds.amazonaws.com
DB_NAME	test
DB_USERNAME	root
DB_PASSWORD	postgres-1
SPRING_PROFILES_ACTIVE	prod (activates production configuration profile)
PORT	8081 (application listening port forwarded by Nginx reverse proxy)
GRADLE_HOME / M2_HOME	/usr/local/gradle / /usr/local/apache-maven

Environment properties
The following properties are passed in the application as environment properties. [Learn more](#)

Name	Value	
GRADLE_HOME	/usr/local/gradle	Remove
M2	/usr/local/apache-maven/bin	Remove
M2_HOME	/usr/local/apache-maven	Remove
DB_HOST_URL	postgres-spring-1.c7sgj40a0vdq.ap-south-1.rds.amazonaws.com	Remove
DB_USERNAME	root	Remove
DB_PASSWORD	postgres-1	Remove
DB_NAME	test	Remove
SPRING_PROFILES_ACTIVE	prod	Remove
PORT	8081	Remove

Add environment property

Cancel Previous **Next**

Figure 21: Environment properties configuration for database connectivity.

07

Step 6: Review Configurations & Launch Environment

Review all settings across the five steps, confirm your deployment preferences, and click **Submit**.

Command timeout 600	Deployment policy Rolling	Health threshold Ok
Ignore health check false	Instance replacement false	
Platform software		
Lifecycle false	Log streaming Deactivated	Logs retention 7
Rotate logs Deactivated	Update level minor	X-Ray enabled Deactivated
Environment properties		
Key	Value	
DB_HOST_URL	postgres-spring-1.c7ygl40a0vdq.ap-south-1.rds.am...	
DB_NAME	test	
DB_PASSWORD	postgres-1	
DB_USERNAME	root	
GRADLE_HOME	/usr/local/gradle	
M2	/usr/local/apache-maven/bin	
M2_HOME	/usr/local/apache-maven	
PORT	8081	
SPRING_PROFILES_ACTIVE	prod	

Cancel Previous **Submit**

Figure 22: Review summary showing rolling deployment policy and environment properties.

Elastic Beanstalk initializes the CloudFormation template, EC2 instances, security groups, and ALB. When complete, the environment status displays **Health: Green**.

Figure 23: Dashboard confirming Green health state and launched EC2 instances.

Click the public environment domain URL to view the default Beanstalk welcome page:

Figure 24: Generated public Beanstalk URL.

Figure 25: Initial sample application live page.

08 Deploying Spring Boot Application JAR Bundle

With the cloud infrastructure running, deploy your compiled Spring Boot executable JAR:

1. From the environment dashboard, click the orange **Upload and deploy** button.
2. Click **Choose file** and select your compiled local JAR (e.g. `library-management-0.0.1-SNAPSHOT.jar`).

3. Assign a **Version label** (e.g., `Library-management-service-version-1`).
4. Set **Deployment preferences** to **Rolling** policy with batch size `1`.
5. Click **Deploy**.

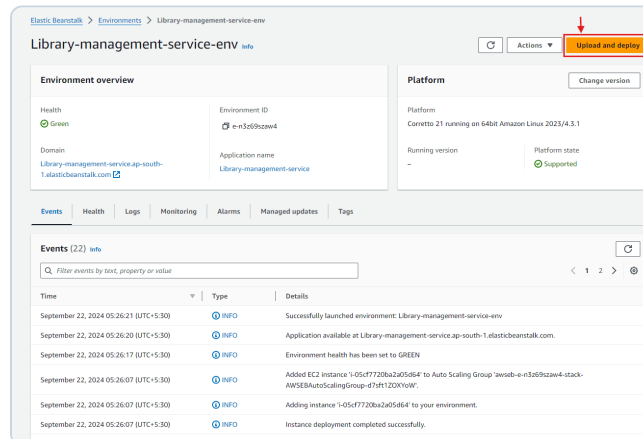


Figure 26: Triggering Upload and Deploy from the Beanstalk overview.

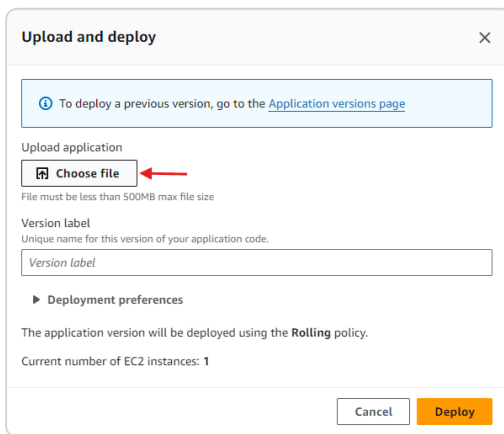


Figure 27: Selecting local application JAR.

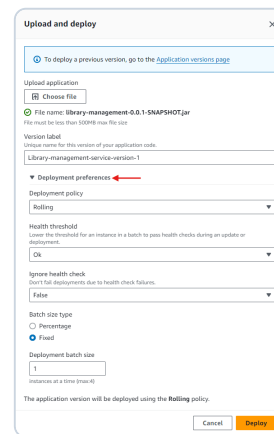


Figure 28: Rolling policy and version label.

09

Deployment Verification & Live Endpoint Validation

Elastic Beanstalk automatically generates a Procfile, replaces the active runtime artifact, and restarts the JVM. The event stream will report: "Instance deployment successfully detected a JAR file in your source bundle" followed by "Instance deployment completed successfully".

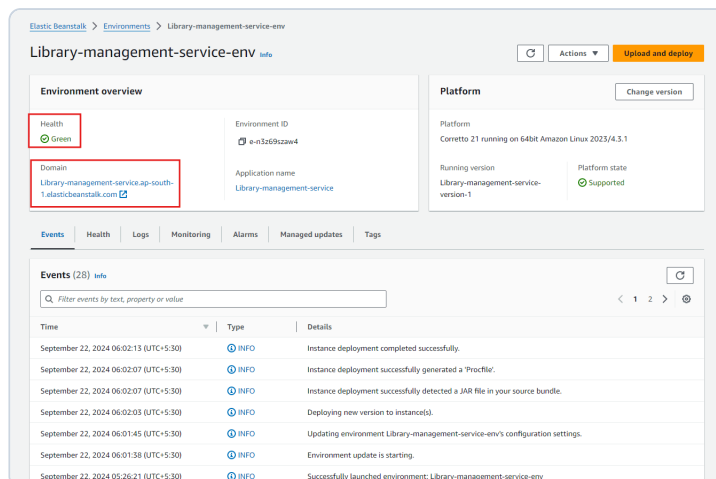


Figure 29: Final dashboard state confirming version-1 is active with Green health.

Navigate to your public Beanstalk domain URL in your browser. The endpoint returns the HTTP 200 string **"Ok"** from your `HealthCheckController`:

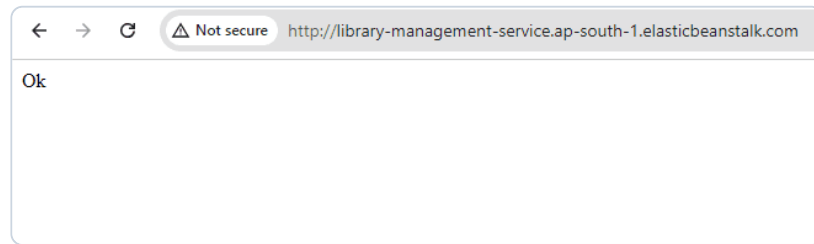


Figure 30: Live browser verification returning 'Ok' from the Spring Boot health controller.

Deployment Successful! Your Spring Boot application is now fully operational on AWS Elastic Beanstalk with high availability, automated CPU-based scaling triggers, Application Load Balancing, and decoupled Amazon RDS database persistence.